VIDYALANKAR INSTITUTE OF TECHNOLOGY

Department of Computer Engineering

Smart Irrigation Alert Dashboard

End-to-End DevOps Project Documentation

(Tasks 1, 2, 3 & 4 Deliverables)

Student Name: Chetan Katkar

Roll Number: 23102A0024

Class & Division: BE CMPN-A

Subject: DevOps Engineering

Academic Year: 2026–2027 (Odd Semester)

Academic Year 2026–2027

Contents

1 Task 1: Problem Definition and Project Scope **2**
1.1 1. Introduction & Background . . . 2
1.2 2. Problem Statement . . . 2
1.3 3. Target Users . . . 2
1.4 4. Existing Pain Points . . . 2
1.5 5. Stakeholders . . . 2
1.6 6. Technical & Project Constraints . . . 3
1.7 7. Measurable Success Criteria . . . 3
1.8 8. Approved Minimum Viable Product (MVP) Scope . . . 3

2 Task 2: Agile Planning and DevOps Workflow **4**
2.1 1. User Stories & Acceptance Criteria . . . 4
2.2 2. Product Backlog . . . 5
2.3 3. Sprint Plan & Definition of Done (DoD) . . . 5
2.4 4. DevOps Lifecycle Architecture . . . 6

3 Task 3: Requirements, Architecture and Technology Setup **7**
3.1 1. Software Requirements Specification (SRS) . . . 7
3.1.1 Functional Requirements (FR) . . . 7
3.1.2 Non-Functional Requirements (NFR) . . . 7
3.2 2. System Architecture . . . 7
3.3 3. REST API Specification . . . 7
3.4 4. Local Development Environment . . . 7

4 Task 4: Git and GitHub SCM Strategy **9**
4.1 1. Repository Structure . . . 9
4.2 2. Branching Strategy & Contribution Workflow . . . 9
4.3 3. Conventional Commit Guidelines . . . 9
4.4 4. Repository Metadata . . . 9

1 Task 1: Problem Definition and Project Scope

1.1 1. Introduction & Background

Agricultural irrigation management is one of the most critical factors influencing crop health and water conservation. Traditional farming relies predominantly on fixed-schedule irrigation or manual in-person inspection of soil moisture. These legacy approaches result in substantial water wastage during over-irrigation or severe crop yield degradation when under-irrigation occurs.

1.2 2. Problem Statement

There is a clear absence of a centralized, real-time, low-latency monitoring platform accessible to small and medium-scale agricultural operations that:

1. Ingests real-time soil moisture and ambient temperature data across multiple farm sectors.
2. Presents telemetry on a responsive, searchable visual dashboard.
3. Automatically raises visual and actionable alerts when soil moisture levels drop below safety thresholds ($< 30\%$).
4. Can be reliably built, tested, packaged, and deployed via a modern continuous DevOps pipeline.

1.3 3. Target Users

#	Target User	Role & Responsibilities
1	**Farmers & Farm Owners**	Monitor daily soil conditions, receive immediate threshold alerts, actuate irrigation cycles.
2	**Agronomists / Consultants**	Evaluate multi-zone historical moisture metrics and recommend precision watering regimes.
3	**DevOps & System Admins**	Maintain continuous delivery pipelines, server health, automated test suites, and container images.

Table 1: Target Users and Operational Roles

1.4 4. Existing Pain Points

- **Manual Inspection Latency:** Physically checking vast acreage is labor-intensive and prone to delayed diagnosis.
- **Water Resource Depletion:** Scheduled timers ignore prevailing weather and actual soil hydration.
- **Lack of Aggregated Telemetry:** No unified single-pane-of-glass dashboard exists across partitioned fields.
- **Manual Deployment Vulnerabilities:** Server configurations and application deployments are traditionally unstandardized and prone to human error.

1.5 5. Stakeholders

- Farmers and agricultural cooperatives (Primary end-users).
- Agronomy researchers and soil test laboratories.

- IoT sensor hardware providers and field automation vendors.
- Water conservation regulatory authorities.

1.6 6. Technical & Project Constraints

- **Software Stack:** Open-source enterprise toolchain (Git, GitHub, Maven, Jenkins, Selenium, Docker, Ansible).
- **Hardware Independence:** IoT sensor feeds are programmatically simulated for reproducible automated testing.
- **Deployment Runtime:** Self-contained executable JAR running on Java 21 LTS with in-memory H2 database.

1.7 7. Measurable Success Criteria

#	Objective	Metric Description	Target
1	Real-time Visualization	Minimum active simulated sensor zones tracked simultaneously	≥ 5 Zones
2	Alert Latency	Time elapsed between threshold breach ($<$ 30%) and UI alert	< 2 seconds
3	Automated Testing	Passing rate of end-to-end Selenium UI regression tests	100% Green
4	Build Automation	End-to-end Jenkins pipeline build and package execution time	< 3 minutes
5	Deployment Portability	Zero-configuration container execution using Docker	Single command

Table 2: Measurable Success Criteria

1.8 8. Approved Minimum Viable Product (MVP) Scope

The MVP is frozen to deliver 5 core capabilities:

1. **Data/Event Entry:** Interactive form to add new agricultural zones with crop types and moisture settings.
2. **Searchable Dashboard:** Live tabular and card views with instantaneous search and status filtering.
3. **Summary Indicators:** Global KPI cards computing Average Moisture, Total Active Zones, and Active Critical Alerts.
4. **Status Drill-Down:** Dedicated modal showing detailed parameters and one-click manual irrigation trigger.
5. **Alert/Exception View:** Automated visual alert banner and badge highlights for any zone below 30% moisture.

2 Task 2: Agile Planning and DevOps Workflow

2.1 1. User Stories & Acceptance Criteria

User Story 1 (US-01): Real-Time Moisture Dashboard

As a Farm Owner, **I want to** view a centralized real-time dashboard displaying soil moisture percentages and ambient temperatures, **So that** I can assess the moisture status of each crop field without physical inspection. **Story Points:** 5 **Priority:** High **Acceptance Criteria:**

- Displays zone name, crop type, moisture level (%), temperature (°C), and status badge.
- Moisture $< 30\%$ displays a red `CRITICAL` badge; 30%–60% displays green `OPTIMAL`; $> 60\%$ displays cyan `SATURATED`.

User Story 2 (US-02): Automated Low-Moisture Alert

As an Agronomist, **I want to** see an immediate visual exception banner whenever moisture drops below 30%, **So that** I can initiate irrigation before crops suffer irreversible dehydration. **Story Points:** 5 **Priority:** Critical **Acceptance Criteria:**

- Top notification banner automatically reflects critical alert count.
- Provides a direct filter shortcut to isolate critical zones immediately.

User Story 3 (US-03): Dynamic Zone Registration

As a Farm Administrator, **I want to** add new sensor zones via an input modal form, **So that** I can expand monitoring as new fields or greenhouses are cultivated. **Story Points:** 3 **Priority:** High **Acceptance Criteria:**

- Form validates required fields (Zone Name, Crop Type, Moisture between 0–100%).
- Submitting adds the record to H2 database and refreshes KPI counters without page reload.

User Story 4 (US-04): Search and Status Filtering

As a Farm Manager, **I want to** filter zones by name and status category, **So that** I can locate specific crops or issues across large acreage. **Story Points:** 3 **Priority:** Medium **Acceptance Criteria:**

- Instant search bar matches zone names or crop types as the user types.
- Dropdown filter allows filtering by `All`, `Critical`, `Optimal`, or `Saturated`.

User Story 5 (US-05): Status Drill-Down and Manual Irrigation Simulation

As an Operator, **I want to** inspect a zone and trigger an immediate irrigation cycle, **So that** I can simulate watering and observe the recovery of soil moisture. **Story Points:** 4 **Priority:** Medium **Acceptance Criteria:**

- Clicking any zone row opens a drill-down dialog with detailed telemetry.
- Clicking "Trigger Irrigation Cycle" sets moisture to 75% and clears the critical alert.

2.2 2. Product Backlog

ID	Item Description	Type	Priority	Points	Sprint
PB-01	Git Repository & Maven Project Boilerplate	SCM	Highest	2	Sprint 1
PB-02	In-Memory H2 DB & Sensor JPA Entity Model	Backend	Highest	3	Sprint 1
PB-03	REST API for Sensor CRUD and IoT Simulation	Backend	High	5	Sprint 1
PB-04	Responsive Web Dashboard with Summary KPI Cards	Frontend	High	5	Sprint 1
PB-05	Critical Low-Moisture ($< 30\%$) Alert Notification	Full-Stack	High	5	Sprint 2
PB-06	Add New Sensor Zone Modal with Validation	Full-Stack	Medium	3	Sprint 2
PB-07	Real-Time Search Bar and Status Filter	Frontend	Medium	3	Sprint 2
PB-08	Zone Drill-Down Modal & Water Trigger	Full-Stack	Medium	4	Sprint 2
PB-09	Selenium WebDriver E2E Automated UI Tests	QA	High	5	Sprint 2
PB-10	Jenkins CI Freestyle Build Job Setup	CI/CD	High	3	Sprint 3
PB-11	Declarative Jenkinsfile Pipeline as Code	CI/CD	High	5	Sprint 3
PB-12	Docker Multi-Stage Image Packaging	Container	High	4	Sprint 3
PB-13	Docker Hub Registry Image Push Pipeline	Container	Medium	3	Sprint 3
PB-14	Ansible Automated Provisioning Playbook	Config Mgmt	Medium	5	Sprint 4
PB-15	Idempotency Validation, Rollback & Final Report	Release	Medium	4	Sprint 4

Table 3: Prioritized Product Backlog

2.3 3. Sprint Plan & Definition of Done (DoD)

- **Sprint 1 (Architecture & Foundation):** Tasks 01–04 (Problem scope, user stories, Spring Boot skeleton, Git repo).
- **Sprint 2 (Core Features & Testing):** Tasks 05–06 & 09–10 (MVP UI, alert banner, merge conflicts, Selenium tests).
- **Sprint 3 (CI/CD & Containerization):** Tasks 07–08 & 11–12 (Jenkins pipeline, Docker containerization, registry push).
- **Sprint 4 (Configuration & Final Release):** Tasks 13–15 (Ansible provisioning, idempotency, documentation).

Definition of Done (DoD) Criteria:

1. Source code compiles cleanly via `mvn clean compile` with zero warnings.
2. All automated Selenium unit and regression tests pass (100% green).
3. Feature branch reviewed and merged into `develop` via verified Pull Request.
4. Application builds into a standalone executable JAR package.
5. Docker container builds and executes cleanly on port 8080.
6. Output logs and screenshots archived in corresponding `Task-XX/` directory.

2.4 4. DevOps Lifecycle Architecture

The project operational workflow adheres to the 7-stage DevOps lifecycle:

1. **Plan:** User stories, acceptance criteria, and backlog tracked via Kanban.
2. **Code:** Java 21, Spring Boot, Git feature branching (`feature/*`).
3. **Build:** Apache Maven compiling classes and assembling the fat JAR.
4. **Test:** Automated Selenium WebDriver UI tests serving as quality gates in Jenkins.
5. **Release / Deploy:** Declarative Jenkinsfile driving multi-stage Docker builds.
6. **Operate:** Ansible playbooks ensuring idempotent environment provisioning.
7. **Monitor:** Live health checks (`/api/sensors`) and threshold alerts.

3 Task 3: Requirements, Architecture and Technology Setup

3.1 1. Software Requirements Specification (SRS)

3.1.1 Functional Requirements (FR)

- **FR-01 (Sensor Ingestion):** System must ingest moisture (0–100%) and temperature (−10°C to 60°C) telemetry.
- **FR-02 (Dashboard View):** System must display all active farm zones in real-time with visual health badges.
- **FR-03 (Summary KPIs):** System must display dynamic cards for Average Moisture, Total Zones, and Critical Alerts.
- **FR-04 (Alert Trigger):** Any zone with moisture $< 30\%$ must automatically trigger a prominent alert banner.
- **FR-05 (Dynamic Search):** Dashboard must support instant client-side text filtering and status filtering.
- **FR-06 (Drill-Down & Watering):** Users must be able to inspect detailed zone telemetry and trigger simulated irrigation.

3.1.2 Non-Functional Requirements (NFR)

- **Performance:** Page response time < 1.5 seconds; REST API turnaround < 200 ms.
- **Reliability:** Zero external database configuration required; embedded H2 database guarantees portability.
- **Maintainability:** Layered architecture following Spring MVC conventions (Controller → Service → Repository → Entity).

3.2 2. System Architecture

The application employs a 4-tier modular architecture:

1. **Presentation Tier:** Responsive Single Page Interface built with HTML5, CSS3, and JavaScript communicating via asynchronous JSON REST calls.
2. **Application Tier:** Spring Boot 3 running on Java 21 LTS, exposing REST controllers (`SensorApiController`) and business logic.
3. **Data Tier:** Spring Data JPA managing an embedded in-memory H2 database.
4. **DevOps & Automation Tier:** Jenkins Pipeline, Selenium WebDriver, Docker Engine, and Ansible Playbooks.

3.3 3. REST API Specification

3.4 4. Local Development Environment

The local development environment has been verified and configured with the following toolchain:

- **Operating System:** Windows 11 64-bit
- **Java Development Kit:** Java SE 21.0.12 LTS
- **Build Tool:** Apache Maven 3.9.9

Method	Endpoint	Description	Status Code
GET	/api/sensors	Retrieve all active sensor zones and readings	200 OK
POST	/api/sensors	Register a new agricultural sensor zone	201 Created
GET	/api/alerts	Fetch list of all zones currently below 30% threshold	200 OK
GET	/api/summary	Compute and return global dashboard KPI statistics	200 OK
POST	/api/simulate	Simulate random moisture and temperature fluctuations	200 OK
POST	/api/sensors/{id}/irrigate	Trigger immediate irrigation cycle for zone (75% moisture)	200 OK

Table 4: REST API Catalog

- **Container Engine:** Docker Desktop 29.6.2
- **Version Control:** Git 2.55.0

4 Task 4: Git and GitHub SCM Strategy

4.1 1. Repository Structure

The repository structure enforces clean separation between application source code and DevOps phase deliverables:

```
smart-irrigation-dashboard/
|-- .github/ISSUE_TEMPLATE/     # Bug and feature request templates
|-- Task-01_Problem-Definition/ # Scope and problem statement docs
|-- Task-02_Agile-Planning/     # User stories, backlog, and sprint plan
|-- Task-03_Architecture-Setup/ # SRS, API catalog, and architecture specs
|-- Task-04_Git-GitHub-Init/    # Branching policies and commit logs
|-- src/main/java/              # Spring Boot Java application code
|-- src/main/resources/static/  # Web UI Dashboard (HTML5/JS/CSS)
|-- pom.xml                     # Maven project descriptor
|-- Dockerfile                  # Container definition (Task 11)
|-- Jenkinsfile                 # Pipeline as Code (Task 8)
`-- README.md                   # Project documentation
```

4.2 2. Branching Strategy & Contribution Workflow

Branch	Role in Workflow	Policy
`main`	Production-ready releases (e.g. tagged `v1.0`)	Protected; merge via PR only
`develop`	Nightly integration branch for finished features	Requires passing build
`feature/*`	Dedicated branches for specific features	Merged via Pull Request
`bugfix/*`	Defect resolution during automated test stages	Merged back to `develop`

Table 5: Git Branching Policy

4.3 3. Conventional Commit Guidelines

All commits throughout the repository follow standardized conventional prefixing:

- `feat:` A new user-facing capability or API endpoint.
- `fix:` A bug or threshold logic correction.
- `docs:` Documentation or markdown modifications.
- `ci:` Changes to `Jenkinsfile`, `Dockerfile`, or pipeline configurations.
- `test:` Adding or adjusting Selenium test suites.

4.4 4. Repository Metadata

- **Project Name:** `smart-irrigation-dashboard`
- **Author:** Chetan Katkar (23102A0024)
- **Primary Branch:** `main`
- **Integration Branch:** `develop`